

Design a Vending Machine

Amazon • Oracle • Adobe • service MNCs – "ఇది ఒక state machine, అంతే" అని చాలా మంది అనుకుంటారు. కానీ ఇక్కడ డబ్బు ఉంది.

మూడు విరుపులు. మొదటిది ఒక అమ్మకాన్ని కారణం లేకుండా తిరస్కరిస్తుంది – 2.14% సార్లు. రెండోది user యొక్క ₹10 మాయం చేస్తుంది. మూడోది డబ్బు తీసుకుని, వస్తువు ఇవ్వకుండా, రెండూ కోల్పోతుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

ఈ PROBLEM యొక్క నిజమైన విషయం

Vending machine ని అందరూ **state machine problem** అనుకుంటారు – IDLE, COLLECTING, DISPENSING. మరియు అది నిజమే, కానీ ఆ భాగం **సులభమైనది**. (ఈ series lo state machines ఇప్పటికే మూడుసార్లు వచ్చాయి – Deep Dive 04, 11, 16.)

ఇక్కడ కొత్తది ఒక్కటే, మరియు అది ముఖ్యమైనది: **డబ్బు**.

డబ్బుకి మూడు లక్షణాలు ఉన్నాయి, మరియు మూడూ ఈ doc lo ఒక్కో విరుపుగా మారతాయి:

- 1 • అది **విడదీయలేనిది** – ₹6 ఇవ్వాలంటే సరైన నాణేలు ఉండాలి (§7)
- 2 • అది **గుర్తింపు కలిగినది** – "₹50" ఒక సంఖ్య కాదు, అది ఒక నిర్దిష్ట నోటు (§10)
- 3 • అది **స్పష్టించబడదు, నాశనం కాదు** – ఏ వైఫల్యంలోనూ ఒక్క రూపాయి కూడా పోకూడదు (§13)

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు state machine ఎందుకు సులభమైన భాగం
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: అత్యాశ చిల్లర

3. Step – పెద్ద నాణెం నుంచి మొదలుపెట్టడం
4. **మొదటి విరుపు** – ₹6 ఇవ్వలేకపోయింది, $3 \times ₹2$ ఉన్నా
5. Step – ఖచ్చితమైన DP • మరియు అది ఎంత ముఖ్యమో కొలవడం

Part 3 – రెండో విరుపు: "50" ఒక సంఖ్య కాదు

6. Step – `inserted += coin`
7. **రెండో విరుపు** – ₹50 ఇచ్చాడు, ₹40 తిరిగి వచ్చింది
8. Step – Escrow • వేసిన నాణేలనే గుర్తుపెట్టుకోవడం

Part 4 – మూడో విరుపు: సగంలో ఆగిపోవడం

9. Step – డబ్బు తీసుకుని, వస్తువు ఇవ్వడం
10. **మూడో విరుపు** – యంత్రం ఇరుక్కుంది, రెండూ పోయాయి
11. Step – పూర్తి rollback

Part 5 – పూర్తి system

12. Step – కొనే ముందు చెప్పడం

13. మొత్తం code · నడిపి చూద్దాం · డబ్బు సంరక్షణ పరీక్ష

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

15. నోటితో చెప్పాల్సిన English script

16. Follow-ups – UPI, పలు కొనుగోళ్ళు, telemetry

17. ఏమి నేర్చుకున్నాం

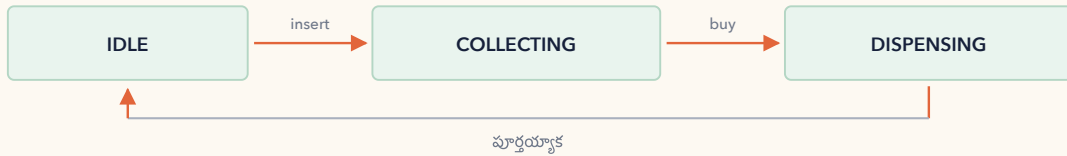
Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు state machine ఎందుకు సులభమైన భాగం

"Design a vending machine. It accepts coins, dispenses items, and returns change."

అందరూ గీసేది · అసలు కష్టం ఎక్కడ ఉందో

State machine సరైనదే – కానీ అది మూడు పంక్తుల్లో ముగుస్తుంది



కానీ ఆ బాణాల మద ఏమి జరుగుతుంది?

"చిల్లర ఇవ్వ" – ఏ నాణేలు? ఉన్నవాటితో అది సాధ్యమేనా? (§4)

"డబ్బు వెనక్కి ఇవ్వ" – ఏ డబ్బు? అతను ఏమి వేశాడో గుర్తుందా? (§7)

"వస్తువు ఇవ్వ" – ఇవ్వలేకపోతే? డబ్బు అప్పటికే తీసుకున్నాం. (§10)

State machine అనేది **ఏ క్రమంలో** అని చెప్పుంది.

ఈ మూడు ప్రశ్నలూ **ఏమి జరుగుతుంది** అని అడుగుతాయి – మరియు అక్కడే డబ్బు పోతుంది.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
ఏ నాణేలు/నోట్లు అంగీకరిస్తుంది?	§5 – చిల్లర సమస్య దీని మీదే ఆధారపడుతుంది

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
యంత్రంలో ఎన్ని నాణేలు ఉంటాయి (float)?	పరిమితం అంటే §4 – ఇదే మొదటి విరుపు
User వేసిన నాణేలు చిల్లరకి వాడొచ్చా?	అవును – మరియు చాలా మంది ఇది మర్చిపోతారు (§8)
చిల్లర ఇవ్వలేకపోతే ఏమి చేయాలి?	§12 – కొనే ముందే చెప్పాలి
వస్తువు ఇరుక్కుంటే?	§11 – మరియు ఇది interviewer అడిగేది
ఒకసారి పలు వస్తువులు?	§16 – escrow design దీన్ని సులభం చేస్తుంది
UPI / కార్డు?	§16 – డబ్బు తీసుకునే విధానం ఒక interface

మూడో ప్రశ్న – మరియు దాదాపు ఎవరూ అడగరు

"User ఇప్పుడే వేసిన నాణేలని, చిల్లర ఇవ్వడానికి వాడొచ్చా?"

జవాబు **అవును** – నిజమైన యంత్రాలు అలాగే చేస్తాయి. User వేసిన ₹10 నాణెం తర్వాతి వ్యక్తికి చిల్లరగా వెళ్ళొచ్చు.

కానీ అది ఒక **సూక్ష్మమైన సమయ ప్రశ్న** లేవనెత్తుతుంది: ఆ నాణేలు **ఎప్పుడు** యంత్రానివి అవుతాయి? అమ్మకం పూర్తయ్యాకనా, వేసిన వెంటనేనా?

జవాబు: **అమ్మకం పూర్తయ్యాకే**. అప్పటిదాకా అవి **escrow** లో ఉంటాయి – యంత్రం దగ్గర ఉన్నాయి, కానీ యంత్రానివి కావు. మరియు ఆ ఒక్క తేడా §7 మరియు §10 – రెండు విరుపులనే పరిష్కరిస్తుంది.

Part 2 – మొదటి విరుపు: అత్యాశ చిల్లర

3. Step – పెద్ద నాణెం నుంచి మొదలుపెట్టడం

చిల్లర ఇవ్వడానికి సహజమైన పద్ధతి: పెద్ద నాణెం నుంచి మొదలుపెట్టి, వీలైనన్ని ఇవ్వడం.

```
// అత్యాశ (greedy): పెద్ద నాణెం నుంచి మొదలుపెట్టి ఇవ్వడం
#makeChange(amount) {
  const give = new Map();
  for (const c of [...this.coins.keys()].sort((a,b) => b-a)) {
    while (amount >= c && (this.coins.get(c) - (give.get(c) ?? 0)) > 0) {
      give.set(c, (give.get(c) ?? 0) + 1);
      amount -= c;
    }
  }
  return amount === 0 ? give : null;
}
```

ఇది మనం నిజంగా చేసేది – ₹37 ఇవ్వాలంటే ఎవరైనా ₹20, ₹10, ₹5, ₹2 అని ఆలోచిస్తారు. మరియు ఇది పనిచేస్తుంది:

```
₹50 వేసి Cola (₹35) కొంటే : { ok: true, item: 'Cola', change: '1×₹10 1×₹5' }
₹20 వేసి Chips (₹20)       : { ok: true, item: 'Chips', change: 'లేదు' }
₹10 వేసి Cola (₹35)        : { ok: false, reason: 'NEED_MORE: 25' }
```

ఒక సాధారణ అపోహ: "భారతీయ నాణేలు {1,2,5,10,20} canonical – కాబట్టి greedy ఎప్పుడూ సరైనది." అది సగం నిజం. నాణేలు **అపరిమితంగా** ఉంటే greedy కనిష్ట సంఖ్యలో నాణేలు ఇస్తుంది.

కానీ ఒక vending machine lo నాణేలు **పరిమితం** – మరియు అక్కడ ఆ హామీ పూర్తిగా చెల్లదు.

4. మొదటి విరుపు – ₹6 ఇవ్వలేకపోయింది, 3×₹2 ఉన్నా

యంత్రంలో ఉన్నవి: **1×₹5, 3×₹2** (₹1 నాణేలు అయిపోయాయి).

```
₹6 చిల్లర ఇవ్వాలి:
  అత్యాశ : సాధ్యం కాదు
  ఖచ్చితం : 3×₹2
₹4 చిల్లర ఇవ్వాలి:
  అత్యాశ : 2×₹2
  ఖచ్చితం : 2×₹2
₹9 చిల్లర ఇవ్వాలి:
```

అత్యాశ : 1×₹5 2×₹2

ఖచ్చితం : 1×₹5 2×₹2

₹6 ఇవ్వడానికి రెండు దారులు · అత్యాశ తప్పుదారి ఎంచుకుంటుంది

యంత్రంలో: 1×₹5, 3×₹2 · ఇవ్వాలి₹6

అత్యాశ

₹5 తీసుకుంది → ₹1 మిగిలింది

₹1 నాణేలు లేవు · ₹2 చాలా పెద్దది

→ "చిల్లర లేదు" · అమ్మకం రద్దు

ఖచ్చితమైనది

₹5 ని **వదిలేసి** → 2+2+2

సరిగ్గా ₹6

→ అమ్మకం పూర్తి ✓

అత్యాశ యొక్క తప్పు: ఒకసారి ₹5 తీసుకున్నాక అది **వెనక్కి తిరగదు**.

అపరిమిత నాణేలు ఉంటే అది సమస్య కాదు – ₹1 ఎప్పుడూ ఉంటుంది.

కానీ ఒక యంత్రంలో నాణేలు **అయిపోతాయి**, మరియు అప్పుడు ఆ ఎంపిక ఒక ఉచ్చు.

మొదటి విరుపు – ఒక అమ్మకాన్ని కారణం లేకుండా తిరస్కరించింది

యంత్రం వద్ద ₹6 ఇవ్వడానికి **సరిపడా నాణేలు ఉన్నాయి**. అది ఇవ్వలేదు.

User కి కనిపించేది: "చిల్లర లేదు, దయచేసి సరిగ్గా డబ్బు వేయండి." అతను వెళ్ళిపోతాడు. **ఒక అమ్మకం పోయింది**, మరియు యంత్ర యజమానికి ఎప్పటికీ తెలియదు.

మౌలిక తప్పు: "చిల్లర ఇవ్వడం" అనేది ఒక విధానం కాదు, ఒక శోధన. "ఏ నాణేల కలయిక ఈ మొత్తానికి సమానం?" అనేది ఒక **సమస్య**, మరియు దానికి ఒక్క దారే ఉందని అనుకోవడం తప్పు.

5. Step – ఖచ్చితమైన DP · మరియు అది ఎంత ముఖ్యమో కొలవడం

"ఏ కలయిక పనిచేస్తుంది?" అనేది ఒక **bounded coin change** problem, మరియు దానికి ఒక సాధారణ DP ఉంది:

```
function makeChange(amount, bag) {
  if (amount === 0) return new CoinBag();
  if (amount < 0) return null;
  const best = new Array(amount + 1).fill(null);
  best[0] = new Map();
  for (const d of bag.denoms) {
    const avail = bag.count(d);
    for (let take = 1; take <= avail; take++)
      for (let v = amount; v >= d; v--)
        if (best[v] === null && best[v-d] !== null &&
            (best[v-d].get(d) ?? 0) < take) {
          const m = new Map(best[v-d]); m.set(d, (m.get(d) ?? 0) + 1); best[v] = m;
        }
  }
  if (best[amount] === null) return null;
  const out = new CoinBag();
  for (const [d, n] of best[amount]) out.add(d, n);
}
```

```
return out;  
}
```

`best[v] = "కేవల ఇవ్వడానికి ఒక చెల్లుబాటుయ్యే నాణేల కలయిక" (లేదా null). ఆ (best[v-d].get(d) ?? 0) < avail` తనిఖీ ఒక నాణేన్ని ఉన్నదానికంటే ఎక్కువసార్లు వాడకుండా ఆపుతుంది.

ఇది నమ్మదగినదేనా? ఒక DP ని కంటితో చూసి నమ్మలేం. కాబట్టి దాన్ని **brute force తో పోల్చాను** – 30,000 యాదృచ్ఛిక సందర్భాలు, అన్ని కలయికలనే ప్రయత్నించే ఒక నెమ్మదైన function తో:

30,000 సందర్భాలు · 28,364 సాధ్యమైనవి

DP = brute force, మరియు ఇచ్చిన నాణేలు చెల్లుతాయా? అవును ✓

ఇది ఎంత తరచుగా పట్టింపు? – కొలిచి చూద్దాం

"అత్యంత కొన్నిసార్లు విఫలమవుతుంది" అనేది ఒక వాదన. **ఎన్నిసార్లు?** అనేది ఒక సంఖ్య:

ఒక్కో నాణెం గరిష్ఠంగా ఎన్ని ఉంటే అత్యంత ఎంత తరచుగా విఫలమవుతుంది

గరిష్ఠం	సాధ్యమైనవి	అత్యంత విఫలం	కోల్పోయిన అమ్మకాలు
0-1	3,404	0	0.00%
0-2	8,565	0	0.00%
0-3	12,522	140	1.12%
0-5	16,635	355	2.13%
0-9	18,904	405	2.14%
0-20	19,765	276	1.40%

ఈ వక్రరేఖ నేను ఊహించనిది

నా అంచనా: **నాణేలు తక్కువైతే అత్యంత ఎక్కువ విఫలమవుతుంది.** కొలత చెప్పినది వేరు.

నాణేలు చాలా తక్కువైతే (0-2): 0% విఫలం. ఎందుకంటే అప్పుడు అత్యంత ఇరుక్కునే సందర్భాలు ఉన్నా, ఆ మొత్తాలని ఏ పద్ధతీ ఇవ్వలేదు – కాబట్టి అవి "కోల్పోయిన అమ్మకాలు" కావు.

నాణేలు సమృద్ధిగా ఉంటే (0-20): 1.40%. ఎందుకంటే అప్పుడు దాదాపు అన్నిటికీ చాలా దారులు ఉంటాయి, అత్యంత ఎంచుకున్నది కూడా పనిచేస్తుంది.

గరిష్ఠం మధ్యలో (0-5 నుంచి 0-9): 2.14% – సరిపడా నాణేలు ఉండి, కానీ అత్యంత తప్పుదారి పట్టడంతో కొరత.

పాఠం: "తక్కువ వనరులు = ఎక్కువ వైఫల్యం" అనే సహజమైన ఊహ ఇక్కడ తప్పు. **సంఖ్య ఊహని తలకిందులు చేసింది** – మరియు అది కొలిచాకే తెలిసింది.

2.14% అంటే ఏమిటి? రోజుకి 200 అమ్మకాలు చేసే యంత్రం – **రోజుకి 4 అమ్మకాలు**, కేవలం చిల్లర లెక్క తప్పు వల్ల.

Part 3 – రెండో విరుపు: "50" ఒక సంఖ్య కాదు

6. Step – inserted += coin

§3 యొక్క code lo డబ్బు ఇలా నిల్వ అవుతుంది:

```
constructor(items, coins) {  
  ...  
  this.inserted = 0;           // ← ఒక సంఖ్య మాత్రమే  
}  
insert(coin) { this.inserted += coin; return this.inserted; }  
refund() { const r = this.inserted; this.inserted = 0; return r; }
```

ఇది సహజమైనది మరియు సరిపోతుంది – ఒక అమ్మకం విజయవంతమైనంత వరకు.

7. రెండో విరుపు – ₹50 ఇచ్చాడు, ₹40 తిరిగి వచ్చింది

యంత్రంలో రెండు ₹20 నాణేలు మాత్రమే. Cola ₹35. User ఒక ₹50 నోటు వేశాడు.

యంత్రం float: 2×₹20. Cola ₹35.

User ఒక ₹50 note వేశాడు: 50

Cola కొనాలంటే : { ok: false, reason: 'NO_CHANGE' }

→ ₹15 చిల్లర ఇవ్వలేదు. సరే, డబ్బు వెనక్కి ఇవ్వాలి.

refund() : ₹50

కానీ – ఏ నోట్లు ఇవ్వాలి? యంత్రానికి తెలియదు.

అది గుర్తుపెట్టుకున్నది "50" అనే సంఖ్య మాత్రమే.

దాని దగ్గర ఉన్నవి: 2×₹20

₹50 ఇవ్వాలంటే 2×₹20 = ₹40 మాత్రమే. ₹10 తక్కువ.

→ User ₹50 note ఇచ్చాడు, ₹40 తిరిగి వచ్చింది. ₹10 మాయం.

రెండో విరుపు - డబ్బుకి గుర్తింపు ఉంది, `INSERTED` కి లేదు

User యొక్క ₹50 నోటు **యంత్రం లోపలే ఉంది**. దాన్ని తిరిగి ఇవ్వడం అతి సులభం.

కానీ code `inserted += 50` అని రాసిన క్షణంలో **ఆ నోటు ఏమిటో మర్చిపోయింది**. ఇప్పుడు "₹50 తిరిగి ఇవ్వు" అంటే అది ఏవైనా నాణేలతో ₹50 కట్టాలి - మరియు అది సాధ్యం కాకపోవచ్చు.

మౌలిక తప్పు: $50 + 20 = 70$ అనేది గణితంలో నిజం. కానీ **ఒక ₹50 నోటు మరియు ఒక ₹20 నోటు అనేవి రెండు నిర్దిష్ట వస్తువులు** - వాటిని కూడితే "₹70" అనే ఒక ముద్ద రాదు.

ఇది Deep Dive 11 §4 (order ఒక live MenuItem ని పట్టుకోవడం) మరియు Deep Dive 15 §4 (ఒక Book = ఒక ప్రతి) - రెండింటికీ బంధువు: **ఒక సంగ్రహణ చాలా ఎక్కువ సమాచారాన్ని పారేసింది.**

8. Step – Escrow · వేసిన నాణేలనే గుర్తుపెట్టుకోవడం

రెండు ముక్కలు:

```
class CoinBag {
  #c = new Map();
  add(d, n = 1) { this.#c.set(d, this.count(d) + n); return this; }
  remove(d, n = 1) {
    if (this.count(d) < n) throw new Error(`NOT_ENOUGH_COINS: ${n}×₹${d}`);
    this.#c.set(d, this.count(d) - n); return this;
  }
  addAll(other) {
    for (const d of other.denoms) this.add(d, other.count(d)); return this;
  }
  removeAll(other) {
    for (const d of other.denoms) this.remove(d, other.count(d)); return this;
  }
  get total() { return this.denoms.reduce((s,d) => s + d*this.count(d), 0); }
}
```

మరియు యంత్రంలో **రెండు సంచులు:**

```
this.float = new CoinBag(float); // యంత్రం సొంత నాణేలు
this.#escrow = new CoinBag(); // user వేసినవి - ఇంకా యంత్రానివి కావు
```

రెండు సంచులు · డబ్బు ఎప్పుడు చేతులు మారుతుంది

Escrow = "యంత్రం దగ్గర ఉండి, కానీ యంత్రానిది కాదు"



చిల్లర లెక్కించేటప్పుడు **రెండు సంచులనీ కలిపి** చూస్తాం –
user ఇప్పుడే వేసిన ₹10 నాణెం, అతనికే చిల్లరగా తిరిగి వెళ్ళిచ్చు.

మరియు **refund ఎప్పుడూ సాధ్యమే** – ఎందుకంటే అతను వేసిన నాణేలే
escrow లో ఉన్నాయి. వాటిని తిరిగి ఇవ్వడానికి ఏ లెక్కా అవసరం లేదు.

```
refund() {  
  if (this.#escrow.isEmpty) return { ok: false, reason: 'NOTHING_TO_REFUND' };  
  const back = this.#escrow; // సరిగ్గా అవే నాణేలు  
  this.#escrow = new CoinBag();  
  this.#state = STATES.IDLE;  
  return { ok: true, returned: back.toString(), total: back.total };  
}
```

ఒక్క లెక్క కూడా లేదు. అతను వేసినవే తిరిగి ఇస్తున్నాం:

```
float 2×₹20, ₹50 వేశాడు: { ok: false, reason: 'NO_CHANGE: ₹15' }  
refund : { ok: true, returned: '1×₹50', total: 50 }  
→ సరిగ్గా అదే ₹50 note తిరిగి వచ్చింది.
```

Part 4 – మూడో విరుపు: సగంలో ఆగిపోవడం

9. Step – డబ్బు తీసుకుని, వస్తువు ఇవ్వడం

ఒక అమ్మకంలో నాలుగు పనులు ఉన్నాయి, మరియు అవి ఈ క్రమంలో జరగాలి:

```
this.float.addAll(this.#escrow);           // 1 • user డబ్బు యంత్రానికి అంది
this.#escrow = new CoinBag();
this.float.removeAll(check.change);        // 2 • చిల్లర బయటికి
s.qty--;                                   // 3 • నిల్వ తగ్గింది
dispenser(s);                             // 4 • వస్తువు బయటికి
```

మొదటి మూడూ **memory lo** జరుగుతాయి – అవి ఎప్పుడూ విజయవంతమవుతాయి. నాలుగోది **భౌతిక ప్రపంచంలో** జరుగుతుంది.

10. మూడో విరుపు – యంత్రం ఇరుక్కుంది, రెండూ పోయాయి

ఒక స్క్రింగ్ ఇరుక్కుంది. Chips బయటికి రాలేదు.

```
before: float 1x₹20 2x₹10 1x₹5 3x₹2 • Chips qty 3
buy → (వస్తువు రాలేదు)
after : float 1x₹20 2x₹10 1x₹5 3x₹2 2x₹20 ... • Chips qty 2
```

మూడో విరుపు – USER ₹20 ఇచ్చాడు, ఏమీ రాలేదు, మరియు యంత్రం "అమ్ముడైంది" అనుకుంటోంది

- 1 • User యొక్క ₹20 float lo కలిసిపోయింది – escrow ఖాళీ, కాబట్టి **refund చేయడానికి ఏమీ లేదు**.
- 2 • qty 3 నుంచి 2 కి తగ్గింది – యంత్రం ఒక Chips అమ్మినట్లు లెక్క.
- 3 • ఆ Chips ఇప్పటికీ లోపలే ఉంది. ఇప్పుడు నిల్వ లెక్క మరియు వాస్తవం శాశ్వతంగా విడిపోయాయి.

మౌలిక తప్పు: మూడు memory మార్పులు ఇప్పటికే జరిగిపోయాయి, ఆపై నాలుగోది విఫలమైంది. మధ్యలో ఆగిపోయిన ఒక ప్రక్రియ – **పాక్షికంగా పూర్తయిన అమ్మకం**.

ఇది database lo అయితే ఒక **transaction**. ఇక్కడా అదే కావాలి: **అన్నీ, లేదా ఏమీ లేదు**.

11. Step – పూర్తి rollback

```
buy(code, { dispenser = () => true } = {}) {
  const check = this.canBuy(code);
  if (!check.ok) return check;

  const s = this.slots.get(code);
  const before = { float: this.float.clone(),
                  escrow: this.#escrow.clone(), qty: s.qty };
  this.#state = STATES.DISPENSING;
  try {
    this.float.addAll(this.#escrow);          // user డబ్బు యంత్రానికి అయింది
    this.#escrow = new CoinBag();
    this.float.removeAll(check.change);      // చిల్లర బయటికి
    s.qty--;
    if (!dispenser(s)) throw new Error(`JAMMED: ${s.name}`);
    this.#state = STATES.IDLE;
    return { ok: true, item: s.name, change: check.change.toString(),
            changeTotal: check.change.total };
  } catch (e) {
    this.float = before.float;                // పూర్తి rollback
    this.#escrow = before.escrow;
    s.qty = before.qty;
    this.#state = STATES.COLLECTING;
    return { ok: false, reason: e.message, refundable: this.#escrow.toString() };
  }
}
```

```
before: float 1x₹20 2x₹10 1x₹5 3x₹2 · Chips qty 3
buy → { ok: false, reason: 'JAMMED: Chips', refundable: '1x₹20' }
after : float 1x₹20 2x₹10 1x₹5 3x₹2 · Chips qty 3
refund: { ok: true, returned: '1x₹20', total: 20 }
```

float అక్షరాలా మారలేదు. qty అలాగే 3. User యొక్క ₹20 తిరిగి వచ్చింది.

ఆ **CLONE()** ఎందుకు, మరియు **DISPENSER** ఒక **PARAMETER** ఎందుకు

clone() – rollback చేయాలంటే "ముందు ఎలా ఉంది" అనేది కావాలి. మార్పులని వెనక్కి లెక్కించడం కంటే ముందటి స్థితిని దాచడం చాలా సులభం మరియు తప్పు చేసే అవకాశం తక్కువ. (సంచులు చిన్నవి – ఇది ఖరీదు కాదు.)

dispenser ఒక parameter – లేకపోతే "ఇరుక్కుంటే ఏమవుతుంది?" అని పరీక్షించలేం. **{ dispenser: () => false }** అని ఇస్తే ఆ దారి పరీక్షించబడుతుంది.

ఇది Deep Dive 14 §12 (సమయం) మరియు Deep Dive 18 §12 (పాచిక) – అదే నియమం: **బయటి ప్రపంచం నుంచి ఏదైనా అడిగితే, దాన్ని బదులుగా ఇవ్వాలి.**

Part 5 – పూర్తి system

12. Step – కొనే ముందు చెప్పడం

§7 lo user ₹50 వేశాడు, ఆపై "చిల్లర లేదు" అని తెలిసింది. అది చాలా ఆలస్యం – అతనికి ముందే తెలియాలి.

```
// కొనే *ముందు* చెప్పగలగడం – "exact change only" సూచిక
canBuy(code, extra = 0) {
  const s = this.slots.get(code);
  if (!s) return { ok: false, reason: `NO_SUCH_ITEM: ${code}` };
  if (s.isEmpty) return { ok: false, reason: `SOLD_OUT: ${s.name}` };
  const paid = this.#escrow.total + extra;
  if (paid < s.price)
    return { ok: false, reason: `NEED_MORE: ₹${s.price - paid}` };
  // చిల్లర float *మరియు* escrow రెండింటి నుంచీ ఇవ్వచ్చు
  const pool = this.float.clone().addAll(this.#escrow);
  const change = makeChange(paid - s.price, pool);
  return change ? { ok: true, change }
    : { ok: false, reason: `NO_CHANGE: ₹${paid - s.price}` };
}
```

మరియు దాని మీద – నిజమైన యంత్రాల మీద కనిపించే ఆ **"EXACT CHANGE ONLY"** దీపం:

```
exactChangeOnly() {
  const out = [];
  for (const s of this.slots.values()) {
    if (s.isEmpty) continue;
    const ok = [...this.accepted].some(d => {
      if (d < s.price) return false;
      return makeChange(d - s.price, this.float) !== null;
    });
    if (!ok) out.push(s.code);
  }
  return out;
}
```

float ఖాళీ → ['D1', 'D2'] కి ఖచ్చితమైన డబ్బు కావాలి
float నిండితే → (ఏదీ లేదు)

canBuy మరియు buy – రెండూ ఒకే తనిఖీలు చేస్తాయి, మరియు buy లోపల canBuy ని పిలుస్తుంది. ఆ సకలు ఉద్దేశపూర్వకం కాదు – అది తొలగించబడింది. UI కి కావాల్సినది "ఇప్పుడు కొంటే ఏమవుతుంది?"; buy కి కావాల్సినది అదే జవాబు. ఒకే logic, రెండు ఉపయోగాలు.

13. మొత్తం code · నడిపి చూద్దాం · డబ్బు సంరక్షణ పరీక్ష

నిర్మాణం · రెండు సంచులు, ఒక పట్టిక, ఒక DP

VendingMachine ఒక సమస్యయకర్త - నిజమైన పని మూడు ముక్కల్లో



అవలం: $\text{float.total} = \text{ప్రారంభం} + \text{వేసినది} - \text{ఇచ్చినది} - \text{ఎల్లప్పుడూ}$.

ఆ ఒక్క సమకరణం jams, refunds, NO_CHANGE - అన్నిటినీ పరీక్షిస్తుంది.

```
'use strict';
class CoinBag {
  #c = new Map();
  constructor(init = {}) {
    for (const [k,v] of Object.entries(init)) this.#c.set(+k, v);
  }
  get denoms() { return [...this.#c.keys()].sort((a,b) => b-a); }
  count(d) { return this.#c.get(d) ?? 0; }
  add(d, n = 1) { this.#c.set(d, this.count(d) + n); return this; }
  remove(d, n = 1) {
    if (this.count(d) < n) throw new Error(`NOT_ENOUGH_COINS: ${n}×₹${d}`);
    this.#c.set(d, this.count(d) - n); return this;
  }
  addAll(o) { for (const d of o.denoms) this.add(d, o.count(d)); return this; }
  removeAll(o) {
    for (const d of o.denoms) this.remove(d, o.count(d)); return this;
  }
  get total() { return this.denoms.reduce((s,d) => s + d*this.count(d), 0); }
  get isEmpty() { return this.total === 0; }
  clone() {
    const b = new CoinBag();
    for (const d of this.denoms) b.add(d, this.count(d));
    return b;
  }
  toString() {
    const p = this.denoms.filter(d => this.count(d))
      .map(d => `${this.count(d)}×₹${d}`);
    return p.length ? p.join(' ') : 'ఎమీ లేదు';
  }
}

function makeChange(amount, bag) {
  if (amount === 0) return new CoinBag();
  if (amount < 0) return null;
  const best = new Array(amount + 1).fill(null);
  best[0] = new Map();
  for (const d of bag.denoms) {
    const avail = bag.count(d);
    for (let take = 1; take <= avail; take++)
      for (let v = amount; v >= d; v--)
        if (best[v] === null && best[v-d] !== null &&
```

```

        (best[v-d].get(d) ?? 0) < avail) {
            const m = new Map(best[v-d]); m.set(d, (m.get(d) ?? 0) + 1); best[v] = m;
        }
    }
    if (best[amount] === null) return null;
    const out = new CoinBag();
    for (const [d, n] of best[amount]) out.add(d, n);
    return out;
}

class Slot {
    constructor(code, name, price, qty) {
        Object.assign(this, { code, name, price, qty });
    }
    get isEmpty() { return this.qty <= 0; }
}

const STATES = { IDLE: 'IDLE', COLLECTING: 'COLLECTING', DISPENSING: 'DISPENSING' };

class VendingMachine {
    #state = STATES.IDLE;
    #escrow = new CoinBag(); // user వేసినవి - ఇంకా యంత్రానివి కావు
    constructor(slots, float, { accepted = [1,2,5,10,20,50] } = {}) {
        this.slots = new Map(slots.map(s => [s.code, s]));
        this.float = new CoinBag(float); // యంత్రం సొంత నాణేలు
        this.accepted = new Set(accepted);
    }
    get state() { return this.#state; }
    get inserted() { return this.#escrow.total; }
    get escrow() { return this.#escrow.toString(); }

    insert(denom) {
        if (this.#state === STATES.DISPENSING) return { ok: false, reason: 'BUSY' };
        if (!this.accepted.has(denom))
            return { ok: false, reason: `NOT_ACCEPTED: ₹${denom}` };
        this.#escrow.add(denom);
        this.#state = STATES.COLLECTING;
        return { ok: true, inserted: this.#escrow.total };
    }

    canBuy(code, extra = 0) {
        const s = this.slots.get(code);
        if (!s) return { ok: false, reason: `NO_SUCH_ITEM: ${code}` };
        if (s.isEmpty) return { ok: false, reason: `SOLD_OUT: ${s.name}` };
        const paid = this.#escrow.total + extra;
        if (paid < s.price)
            return { ok: false, reason: `NEED_MORE: ₹${s.price - paid}` };
        const pool = this.float.clone().addAll(this.#escrow);
        const change = makeChange(paid - s.price, pool);
        return change ? { ok: true, change }
            : { ok: false, reason: `NO_CHANGE: ₹${paid - s.price}` };
    }

    buy(code, { dispenser = () => true } = {}) {
        if (this.#state === STATES.DISPENSING) return { ok: false, reason: 'BUSY' };
        const check = this.canBuy(code);
        if (!check.ok) return check;

        const s = this.slots.get(code);

```

```

const before = { float: this.float.clone(),
                  escrow: this.#escrow.clone(), qty: s.qty };
this.#state = STATES.DISPENSING;
try {
  this.float.addAll(this.#escrow);
  this.#escrow = new CoinBag();
  this.float.removeAll(check.change);
  s.qty--;
  if (!dispenser(s)) throw new Error(`JAMMED: ${s.name}`);
  this.#state = STATES.IDLE;
  return { ok: true, item: s.name, change: check.change.toString(),
          changeTotal: check.change.total };
} catch (e) {
  this.float = before.float;           // పూర్తి rollback
  this.#escrow = before.escrow;
  s.qty = before.qty;
  this.#state = STATES.COLLECTING;
  return { ok: false, reason: e.message, refundable: this.#escrow.toString() };
}
}

refund() {
  if (this.#escrow.isEmpty) return { ok: false, reason: 'NOTHING_TO_REFUND' };
  const back = this.#escrow;           // సరిగ్గా అవే నాణేలు
  this.#escrow = new CoinBag();
  this.#state = STATES.IDLE;
  return { ok: true, returned: back.toString(), total: back.total };
}

exactChangeOnly() {
  const out = [];
  for (const s of this.slots.values()) {
    if (s.isEmpty) continue;
    const ok = [...this.accepted].some(d =>
      d >= s.price && makeChange(d - s.price, this.float) !== null);
    if (!ok) out.push(s.code);
  }
  return out;
}
}

```

--- సాధారణ కొనుగోలు ---

```

float : 1×₹20 2×₹10 1×₹5 3×₹2
₹50 వేశాక escrow : 1×₹50 · state COLLECTING
Cola (₹35) : { ok: true, item: 'Cola', change: '1×₹10 1×₹5', changeTotal: 15 }
float ఇప్పుడు : 1×₹50 1×₹20 1×₹10 3×₹2

```

--- §7: అత్యాశ విఫలమయ్యే చోటు ---

```

float: 1×₹5 3×₹2 · Bar ₹14
₹20 వేశాక - ₹6 చిల్లర కావాలి
అత్యాశ అయితే: ₹5 తీసుకుని, ₹1 దొరకదు → విఫలం
ఖచ్చితమైన DP : { ok: true, item: 'Bar', change: '3×₹2', changeTotal: 6 }

```

--- §10: చిల్లర ఇవ్వలేనప్పుడు ---

```

float 2×₹20, ₹50 వేశాడు: { ok: false, reason: 'NO_CHANGE: ₹15' }
refund : { ok: true, returned: '1×₹50', total: 50 }

```

```

--- యంత్రం ఇరుక్కుంటే (jam) ---
before: float 1x₹20 2x₹10 1x₹5 3x₹2 · Chips qty 3
buy → { ok: false, reason: 'JAMMED: Chips', refundable: '1x₹20' }
after : float 1x₹20 2x₹10 1x₹5 3x₹2 · Chips qty 3
refund: { ok: true, returned: '1x₹20', total: 20 }

--- తిరస్కరణలు ---
అంగీకరించని నాణెం : { ok: false, reason: 'NOT_ACCEPTED: ₹3' }
డబ్బు లేకుండా      : { ok: false, reason: 'NEED_MORE: ₹20' }
సరిపోని డబ్బు     : { ok: false, reason: 'NEED_MORE: ₹25' }
అమ్ముడుపోయినది : { ok: false, reason: 'SOLD_OUT: Water' }
లేని code         : { ok: false, reason: 'NO_SUCH_ITEM: Z9' }
ఖాళీ refund       : { ok: false, reason: 'NOTHING_TO_REFUND' }

--- "ఖచ్చితమైన డబ్బు మాత్రమే" సూచిక ---
float ఖాళీ → [ 'D1', 'D2' ] కి ఖచ్చితమైన డబ్బు కావాలి
float నిండితే → (ఏదీ లేదు)

```

మరియు అసలు రుజువు – డబ్బు సంరక్షణ

ఒక vending machine కి ఒకే ఒక అచలం (invariant) ముఖ్యమైనది:

float.total = ప్రారంభ float + user వేసినది – user కి తిరిగి వెళ్ళినది

అది ఎప్పుడూ నిజమై ఉండాలి – విజయవంతమైన అమ్మకాల్లోనూ, jams lo నూ, refunds lo నూ, NO_CHANGE lo నూ. **3,000 యాదృచ్ఛిక యంత్రాలు, ఒక్కొక్కటి 12 యాదృచ్ఛిక operations:**

```

3,000 యంత్రాలు · 2338 అమ్మకాలు · 385 jams · 3249 refunds · 500 NO_CHANGE
డబ్బు సంరక్షణ + escrow + నాణేల లెక్క + వస్తువుల లెక్క: అన్నీ సరైనవి ✓
makeChange: 16,620 ఫలితాలు · మొత్తం సరైనదా, నిల్వ దాటలేదా: అవును ✓

```

నాలుగు అచలాలు పరీక్షించాం: డబ్బు సృష్టించబడలేదు/నాశనం కాలేదు, escrow చివరికి ఖాళీ, ఏ నాణెం లెక్కా ముణాత్మకం కాదు, మరియు బయటికి వెళ్ళిన వస్తువులు = qty తగ్గుదల.

ఒక సాధారణ vending machine పరీక్ష ఎలా ఉంటుంది? – "₹50 వేసి ₹35 వస్తువు కొంటే ₹15 రావాలి." అది ఒక సందర్భం.

ఒక అచలం వేరు: అది ప్రతి సందర్భాన్నీ పరీక్షిస్తుంది. 385 jams – ఒక్కొక్కటి వేరే స్థితిలో – అన్నీ డబ్బు సమీకరణాన్ని గౌరవించాయి. నేను ఆ 385 సందర్భాలని **రాయలేదు**; నేను ఒక **నియమం** రాశాను.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	అత్యాశ చిల్లర	మౌలిక అస్థిపంజరం
§4 (విరుపు)	makeChange DP	₹6 ఇవ్వలేకపోయింది, 3x₹2 ఉన్నా
§5	DP vs brute force	ఒక DP ని కంటితో నమ్మలేం

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§7 (విరుపు)	<code>CoinBag</code> + escrow	₹50 ఇచ్చాడు, ₹40 వచ్చింది
§8	చిల్లరకి float + escrow	User వేసిన నాణేలూ వాడొచ్చు
§10 (విరుపు)	<code>clone()</code> + rollback	Jam lo డబ్బు మరియు వస్తువు రెండూ పోయాయి
§11	<code>dispenser</code> ఒక parameter	లేకపోతే jam ని పరీక్షించలేం
§12	<code>canBuy</code> , <code>exactChangeOnly</code>	User కి ముందే తెలియాలి
§13	డబ్బు సంరక్షణ అచలం	సందర్భాలు కాదు, ఒక నియమం

Part 6 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

State machine ని వేగంగా దాటండి – డబ్బే అసలు విషయం

6 నిమి

Clarify – float పరిమితమా? escrow వాడొచ్చా? jam?

7 నిమి

States + slots వేగంగా

13 నిమి – చిల్లర

అత్యంత ఎక్కువ విరుగుతుంది · DP

11 నిమి – Escrow

refund · rollback డబ్బు సంరక్షణ

8 నిమి

canBuy · UPI · telemetry

ఏమి తప్పక చెప్పాలి

1. "Float పరిమితమా?" అని అడగండి (§2) – అపరిమితం అంటే greedy సరిపోతుంది; పరిమితం అంటే §4.
2. Greedy ఎక్కువ విరుగుతుందో ఒక ఉదాహరణతో (§4) – "1×₹5, 3×₹2, ₹6 కావాలి." ఇది 20 సెకన్లు మరియు ఇది నిర్ణయాత్మకం.
3. Escrow ఒక విడి సంచి (§8) – "the coins aren't the machine's until the sale completes." ఇది refund ని లెక్క లేని పని చేస్తుంది.
4. చిల్లర float + escrow నుంచి (§8) – చాలా మంది ఇది మర్చిపోతారు.
5. Jam → పూర్తి rollback (§11), మరియు డబ్బు సంరక్షణ అచలం (§13). ఈ రెండూ కలిసి "నేను డబ్బు ఉన్న system రాస్తున్నానని నాకు తెలుసు" అని చెప్తాయి.

ఏమి వదిలేయాలి

- CoinBag పూర్తిగా రాయొద్దు – "a multiset of denominations with add/remove" ఒక వాక్యం.
- DP ని రాయాలా? – recurrence చెప్పి, "it's bounded coin change" అని పేరు పెట్టి, సమయం ఉంటే రాయండి.
- exactChangeOnly – ప్రస్తావించండి, రాయొద్దు.
- State machine మీద ఎక్కువ సమయం పెట్టొద్దు – మూడు states, మూడు బాణాలు, ముందుకీ.

15. నీటితో చెప్పాల్సిన English script

"Three questions first. Is the machine's coin float limited, or can I assume it always has change? Can I use the coins the customer just inserted to make their change? And what happens if the item jams after they've paid?

The state machine part – idle, collecting, dispensing – is about three lines, so I'd rather spend the time on the money.

The obvious way to make change is greedy: biggest coin first. With unlimited coins and Indian denominations that's actually optimal, so it feels safe. But a vending machine's float is limited, and there greedy doesn't just give a worse answer – it fails to find one that exists. If the machine holds one five-rupee coin and three two-rupee coins and owes six rupees, greedy takes the five, needs one more, has no ones, and reports no change available. Three twos would have worked perfectly. The customer walks away and the owner never finds out.

I measured how often that matters, because 'sometimes greedy fails' isn't a design argument. Over twenty thousand random floats it peaked at about two point one percent of otherwise-servable sales. Interestingly it isn't monotonic – with very few coins it's zero percent, because the cases where greedy gets stuck are also cases nobody could serve. It's worst in the middle, where there are enough coins to have options but few enough to strand yourself. At two hundred sales a day that's four lost sales daily, so I'd use the exact version: bounded coin change, a small DP over the amount. And I'd verify it against brute force, because a DP you can't check is a DP you can't trust.

Second thing, and this one is easy to miss. If you track inserted money as a running total, you've thrown away which notes the customer gave you. Then when you owe them a refund, you have to construct fifty rupees out of whatever's in the float, and that can be impossible – I had a case where the customer inserted a fifty and got forty back. So the inserted coins go into an escrow: a separate bag, physically in the machine but not yet the machine's money. Refund is then handing back the exact same coins, with no computation at all and no failure mode. And when computing change I'd draw from float plus escrow together, because the coin they just inserted is available.

Third: the escrow only merges into the float when the sale completes. That matters because the last step – actually dispensing – is the one that happens in the physical world and can fail. If the money has already moved and the quantity has already decremented, a jam means the customer paid, got nothing, and the inventory count is now permanently wrong. So I'd snapshot the float, escrow and quantity before the dispense, and roll all three back if it fails. It's a transaction.

For testing, I wouldn't write individual cases. There's one invariant: the float's total always equals the starting float plus what went in minus what came out. I ran three thousand random machines through random sequences of inserts, buys, jams and refunds and checked that equation, plus that escrow ends empty, no coin count goes negative, and items dispensed matches the drop in

quantity. One rule covers every case, including the three hundred and eighty-five jams I never had to write by hand.

Last, I'd expose a canBuy that runs the same checks without committing, so the display can show 'exact change only' before the customer inserts anything rather than after."

16. Follow-ups – UPI, పలు కొనుగోళ్ళు, telemetry

Follow-up	జవాబు	మారే classes
"ఒకేసారి రెండు వస్తువులు"	Escrow ఇప్పటికే మొత్తం పట్టుకుంటుంది – <code>buy</code> ని ఒక జాబితా తీసుకునేలా	<code>buy</code>
"UPI / కార్డు"	<code>PaymentSource</code> interface – cash ఒక అమలు	+1 కొత్తది
"ధరలు మారాలి"	<code>Slot.price</code> ఒక field – కానీ escrow ఉండగా మార్చేద్దు	0
"నాణేలు నిండిపోతే?"	<code>CoinBag</code> కి ఒక capacity; నిండితే ఆ denomination తిరస్కరించడం	<code>insert</code>
"ఏ వస్తువు ఎక్కువ అమ్ముడైంది?"	Deep Dive 19 యొక్క hit counter – <code>bump (code)</code>	0 concepts
"ఎప్పుడు నింపాలో తెలియాలి"	కింద చూడండి	Telemetry layer
"ఇద్దరు ఒకేసారి"	కింద చూడండి	–

ఒకేసారి ఇద్దరు – మరియు ఇక్కడ ఒక ఊరట

"ఈ series lo చాలాసార్లు races చూశాం (Deep Dive 04 §7, 15 §16). ఇక్కడ మాత్రం అది **చాలా సులభం** – ఎందుకంటే ఒక భౌతిక యంత్రానికి **ఒక్కటే నాణెం రంధ్రం** ఉంది.

ఒక సమయంలో ఒక్క user మాత్రమే. కాబట్టి `DISPENSING` స్థితిలో `insert` ని తిరస్కరిస్తే చాలు – అదే మొత్తం lock.

కానీ ఇది మారే చోటు ఒకటి ఉంది: ఒక cloud-connected యంత్రం. నిల్వని ఒక server కి report చేస్తుంటే, మరియు app నుంచి ముందే reserve చేసుకోగలిగితే – అప్పుడు Deep Dive 04 యొక్క atomic claim మళ్ళీ అవసరం."

Telemetry – మరియు అది design ని ఎలా మారుస్తుంది

"'ఎప్పుడు నింపాలి?' అనేది ఒక inventory ప్రశ్న, కానీ **రెండు రకాల నిల్వ ఉన్నాయి**: వస్తువులు మరియు **నాణేలు**.

వస్తువులు అయిపోతే అమ్మకం ఆగుతుంది – స్పష్టం. కానీ **చిల్లర నాణేలు అయిపోతే** యంత్రం "పని చేస్తూనే" ఉంటుంది, కేవలం **ఎక్కువ అమ్మకాలని తిరస్కరిస్తూ**. అది dashboards lo కనిపించదు.

అందుకే నేను `exactChangeOnly()` ని ఒక method గా బయటపెట్టాను – అది UI సూచిక మాత్రమే కాదు, **ఒక telemetry సంకేతం**: "ఈ యంత్రానికి చిల్లర కావాలి, ఇంకా ఏమీ అయిపోకముందే."

§5 యొక్క 2.14% ని గుర్తుంచుకోండి – **కనిపించని నష్టాలు కొలవకపోతే ఎప్పటికీ కనిపించవు.**"

17. ఏమి నేర్చుకున్నాం

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
అత్యాశ యొక్క దాచిన ఊహ	అపరిమిత నాణేలు (§4)	Deep Dive 16 §7 · ప్రతి "తెలివైన" ఉపాయం
ఊహని కొలవడం	2.14%, మరియు వక్రరేఖ (§5)	Deep Dive 05 §10, 03 §8, 19 §11
గుర్తింపు ఉన్నదాన్ని మొత్తంగా మార్చేద్దు	<code>inserted += coin</code> (§7)	Deep Dive 11 §4, 15 §4
Escrow – "నా దగ్గర ఉంది ≠ నాది"	float vs escrow (§8)	Payments, bookings, two-phase commit
అంతా లేదా ఏమీ లేదు	Jam rollback (§11)	Transactions, sagas, Deep Dive 04 §7
బయటి ప్రపంచాన్ని inject చెయ్యి	<code>dispenser</code> (§11)	Deep Dive 14 §12, 18 §12
సందర్భాలు కాదు – ఒక అచలం	డబ్బు సంరక్షణ (§13)	ప్రతి ఆర్థిక system

ఒక చివరి ఆలోచన – ఈ DOC LO నా ఊహ తప్పిన చోటు

§5 lo నేను అత్యాశ ఎంత తరచుగా విఫలమవుతుందో కొలిచాను, మరియు **నా అంచనా స్పష్టంగా తప్పు** అని తేలింది. నేను "నాణేలు తక్కువైతే ఎక్కువ విఫలం" అనుకున్నాను. వాస్తవం ఒక **తలకిందుల U** – అతి తక్కువ దగ్గర 0%, మధ్యలో 2.14%, సమృద్ధి దగ్గర 1.40%.

ఒక్కసారి ఆలోచిస్తే కారణం స్పష్టం: **అత్యాశ విఫలం కావాలంటే ముందు ఒక జవాబు ఉండాలి**. నాణేలు చాలా తక్కువైతే జవాబే ఉండదు, కాబట్టి "కోల్పోయిన అమ్మకం" అనేదే లేదు.

కానీ అది ఆలోచించి రాబట్టినది కాదు – **కొలిచి తెలుసుకున్నది**. మరియు ఈ series అంతటా అదే నమూనా: **ఒక సంఖ్య ఒక ఊహని తలకిందులు చేసినప్పుడు, ఆ క్షణమే నిజంగా ఏదో నేర్చుకున్నట్టు**.

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 Cache · 03 Rate Limiter · 04 BookMyShow · 05 Splitwise · 06 Elevator · 07 Pub-Sub · 08 HashMap · 09 Chess · 10 Meeting Scheduler · 11 Food Delivery · 12 File System · 13 Leaderboard · 14 Text Editor · 15 Library · 16 Tic-Tac-Toe · 17 Autocomplete · 18 Snake & Ladder · 19 Hit Counter · 20 Vending Machine.**

Deep Dive 04 (BookMyShow) దీనికి దగ్గరి బంధువు – అక్కడా డబ్బు, అక్కడా "పాక్షికంగా పూర్తయిన" ప్రమాదం, అక్కడా ఒక escrow-లాంటి HELD స్థితి.

Vending Machine – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసి తీసినదే 